Custom Object Detection for Text Symbols: A YOLO Baseline and a From-Scratch CNN Comparison

[Your Name]

[Matriculation Number]

College of Computing and Data Science

Nanyang Technological University, Singapore

Abstract—This report presents a custom object detection system for text symbols, built on a YOLOv8n detector adapted by transfer learning from COCO-pretrained weights to a 29-class dataset of photographed symbol cards. The dataset was captured, annotated and verified as part of the work, comprising 27 multi-symbol photographs containing 323 annotated instances. The fine-tuned detector reaches mAP@0.5 of 0.914 and mAP@0.5:0.95 of 0.894 on a held-out split, with markedly uneven per-class recall. Beyond the required baseline, a second, independent pipeline was implemented for comparison: a classical segmentation stage (colour-invariant thresholding and convex-hull reconstruction) feeding a small convolutional classifier trained from scratch. Evaluating both on an identical held-out set, and separating detection from classification rather than reporting one blended figure, shows the classical detector recovering 100% of instances against YOLO's 84.7%, while the dedicated classifier reads correctly-detected symbols slightly more accurately than YOLO's own head (94.4% versus 90.2%). Testing under adverse conditions reverses expectations further: on three photographs taken against previously unseen backgrounds, the fine-tuned detector produced no detections at all, while the hand-specified geometric rule succeeded on two. The results are interpreted as a consequence of limited training diversity rather than of architecture, and an inappropriate default augmentation setting is identified as a likely contributor.

Keywords—object detection, YOLO, transfer learning, convolutional neural networks, symbol recognition, data augmentation, failure analysis

I. Introduction

Object detection differs from image classification in that a detector must answer two questions simultaneously: what objects are present, and where each one is. This assignment concerns the detection of text symbols—digits, letters and a small number of pictographic symbols—printed individually on black cards and photographed in ordinary environments. For every symbol in an image the system must report a class label, a confidence score, and bounding-box coordinates.

The required baseline is a YOLO detector adapted by transfer learning from a pretrained checkpoint. That baseline was implemented and forms the core of this report. However, the assignment places explicit emphasis on experimental work and analysis rather than on peak accuracy, and the most informative results here came from comparing the learned detector against a deliberately different approach and from testing both outside the conditions they were trained on.

The contributions of this work are as follows. First, a dataset of photographed symbol cards was captured, annotated, and verified, with the annotation pipeline itself made reproducible. Second, a YOLOv8n detector was fine-tuned and evaluated with standard detection metrics. Third, an independent comparison system—classical segmentation feeding a small convolutional classifier trained from scratch—was built and evaluated on the identical held-out data, with detection and classification performance reported separately so that the source of each error is visible. Fourth, several failure modes were diagnosed to root cause rather than merely observed, including one caused by camera flash and one caused by an augmentation setting that is inappropriate for symbol data. Fifth, a measurement methodology is adopted—repeated training runs under different random seeds—after single-run results were shown to be unreliable at this dataset scale.

II. Dataset

A. Data Collection

The dataset consists of photographs of a physical set of symbol cards. Each card is a matte black square bearing a single symbol printed in one of five colours. The full symbol inventory comprises the digits 1 to 9, the letters A to H and S to Z, and four pictographic symbols: a filled circle and left-, right- and up-pointing arrows, giving 29 classes in total. This exceeds the required minimum of ten classes.

Photographs were captured in three sessions using a mobile phone camera under ordinary indoor lighting. The sessions deliberately varied the conditions: cards were photographed on a clipboard, on two different marble surfaces, and on a woven mat, from a range of heights and viewing angles, with individual cards left rotated by up to roughly twenty degrees. A later session spaced the cards further apart after an earlier attempt with cards placed edge to edge proved problematic for annotation, as described below. Approximately ninety photographs were captured in total, of which twenty-seven are used for detector training and evaluation; the remainder were either reserved, used for the comparison classifier, or excluded for reasons documented in Section II-E.

B. Annotation Methodology

Annotating bounding boxes by hand for several hundred instances would have been slow and error-prone. Instead, annotation was semi-automated. A deterministic classical detector locates candidate cards in a photograph by thresholding for pixels that are simultaneously dark and low in saturation—the signature of the black card material as distinct from coloured symbols or ordinary backgrounds—then grouping those pixels into connected components and filtering by area and aspect ratio. Each candidate is cropped and assembled into an indexed contact sheet, from which the true class of every candidate is read off visually and recorded. Because the detector is deterministic, re-running it on the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

same photograph reproduces the same candidates in the same order, so box coordinates can be regenerated from the recorded index-to-label maps without re-annotating anything.

This has a useful property for reproducibility: the labelling decisions are stored as explicit, auditable maps in source control rather than as opaque coordinate files, and the script that converts them into YOLO-format annotations is included in the repository.

C. Dataset Composition

TABLE I. DATASET COMPOSITION

Property	Value
Classes	29 (25 characters, 4 pictographs)
Images (train)	18
Images (validation)	9
Annotated instances (train)	251
Annotated instances (val)	72
Instances per image	6 to 28
Image resolutions	1200x1600, 1600x1200, 1536x2048, 2048x1536
Annotation format	YOLO (class, cx, cy, w, h; normalised)
Split strategy	By source photograph, not by instance

The split is by photograph rather than by instance. This matters: splitting individual boxes at random would place crops of the same physical card in the same lighting and pose on both sides of the split, which would inflate validation scores. Every validation photograph is one the detector has never seen in any form.

Class distribution is uneven, and unavoidably so, because each photograph contains whichever subset of the deck was laid out at the time. In the validation split, classes such as T, U, V, W, X, Y and Z appear four to five times each, while several digits appear only once or twice. This imbalance is revisited in Section V when interpreting per-class recall, where a single missed instance in a one-instance class moves that class's recall from 1.0 to 0.0.

D. Annotation Verification

The assignment requires that annotations be verified before training. Every generated annotation file was rendered back onto its source photograph and inspected. Figure 1 shows one such verification image.

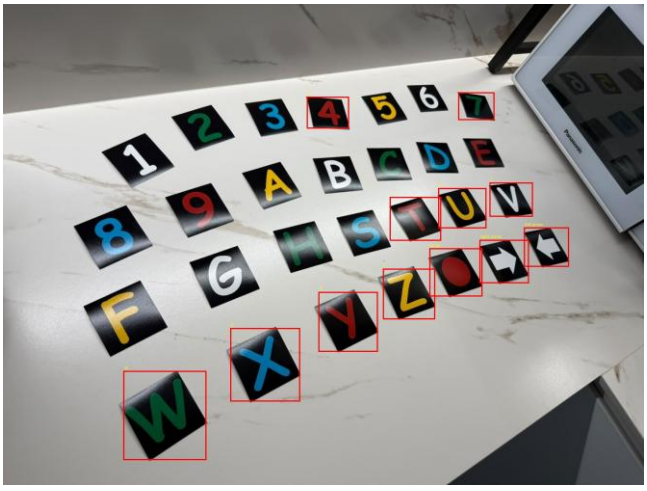


Fig. 1. Annotation verification. Generated YOLO boxes rendered back onto a training photograph. All twelve annotated instances in this image are correctly placed and labelled; unannotated cards in the frame belong to a different source-photograph subset.

This step caught a real defect. An initial inspection at reduced resolution appeared to show missing boxes, which prompted a check of the underlying label files; the labels were in fact complete, and the apparent absence was an artefact of the preview scale. The lesson retained was that visual verification must be performed at a resolution where the smallest annotation is legible.

E. Data Quality Defects Discovered

A more consequential defect surfaced later. The comparison classifier described in Section III-B is trained on individual card crops, and an audit of every crop's segmentation output—not merely its pixel statistics—revealed that nine of them were silently corrupted. In some, a neighbouring card had fused with the target; in others, a shadowed surface adjacent to the card had merged with it during thresholding, so that the reconstructed card outline extended well beyond the physical card and swept background into the symbol mask.

Four of these came from the very first batch of crops, and had gone undetected through several rounds of training. The reason is instructive: they had been checked only by pixel count, a statistic that cannot distinguish a correct symbol from an incorrect shape of similar area. They also could not be caught by disagreement, because at that stage each class had exactly one example, so a corrupt example simply became that class's definition. The defect only became visible once multiple genuine examples per class existed and could contradict one another. Two further sources were excluded wholesale: a photograph taken on a dark, patterned mat, whose background repeatedly defeated the black-card threshold, and a recurring oversized crop of one letter.

III. METHODOLOGY

A. YOLO Baseline with Transfer Learning

The baseline detector is YOLOv8n, the smallest model in the YOLOv8 family, implemented with the Ultralytics framework [2]. Training starts from the publicly released COCO-pretrained checkpoint rather than from random initialisation, as required. The rationale for expecting transfer to help is specific: the low-level features a detector needs—edges, corners, closed contours, figure-ground separation—are largely domain-independent, so a backbone pretrained on a large natural-image detection corpus [3] begins with a substantial head start on locating a small rectangular object, even though no COCO class resembles a symbol card. Only the detection head must learn the new 29-class vocabulary.

TABLE II. YOLO TRAINING CONFIGURATION

Setting	Value
Model	YOLOv8n (COCO-pretrained)
Parameters	3,011,303 (fused), 72 layers
Compute	8.1 GFLOPs
Input image size	640 x 640
Classes	29
Epochs	150 (patience 40)
Batch size	8
Optimizer	AdamW (auto-selected)
Initial learning rate	0.000303
Momentum	0.9
Weight decay	0.0005
Random seed	42
Hardware	Intel Core i5-12450HX (CPU)

Setting	Value
Training time	744 s (12.4 min)

One configuration detail is worth recording because it was not intended. Although an NVIDIA RTX 3050 with 6 GB of memory was available, training ran on CPU. Installing the Ultralytics package into the shared environment replaced the CUDA build of PyTorch with a CPU-only build without warning, and the substitution was noticed only when the training log reported the device. Training was fast enough at this dataset size that the run was not repeated, but the incident is documented because it is exactly the kind of silent environment change that invalidates timing comparisons.

B. Comparison System: Segmentation and a From-Scratch CNN

The second pipeline separates the two questions that YOLO answers jointly. Localisation is performed by the classical detector already described. Classification is performed by a small convolutional network trained from scratch on individual card crops.

The segmentation stage is worth describing because two of its design decisions were forced by failures. A card photographed at an angle is a rotated square, and the axis-aligned bounding box of a rotated square necessarily contains background in its four corners; an early version that treated everything non-black inside that box as symbol therefore swept those corners into the mask and severely distorted tilted cards. Filling the interior of the black region instead fails differently: some symbols, such as the digit 6, have a stroke that reaches the physical edge of the card, so the interior is not an enclosed region and nothing is filled. The resolution used throughout is to take the convex hull of the card's black pixels. A physical card is convex, so its hull is its true footprint regardless of whether its outline happens to be closed, and both failure modes disappear at once.

The classifier receives a 64x64 binary mask—symbol versus not-symbol—rather than the colour crop. This is deliberate. In the source deck each symbol happens to be printed in a consistent colour, so a network given colour could reach high accuracy by learning the colour-to-class mapping while learning nothing about shape, and would fail immediately on a differently coloured card. Discarding colour before training removes the shortcut. The network itself has three convolutional blocks (16, 32 and 64 channels, each with batch normalisation, ReLU and 2x2 max pooling) followed by a fully connected head, totalling approximately 551,000 parameters. It is trained on synthetic variants generated on the fly from the real crops by random rotation, scaling, translation, stroke thickening or thinning, and additive noise.

C. Evaluation Protocol

Detection performance is reported with precision, recall, mAP@0.5 and mAP@0.5:0.95. For the pipeline comparison, a further distinction is drawn that a single accuracy figure would hide. A prediction is matched to a ground-truth instance greedily by intersection over union at a threshold of 0.5. Detection recall is then the proportion of real symbols matched by any prediction; classification accuracy given detection is the proportion of matched symbols whose predicted class is also correct; and overall accuracy requires both. A system can fail by not finding a symbol or by finding it and misreading it, and these demand different remedies.

For the classifier, every reported figure is the mean over five independent training runs with different random seeds, with the standard deviation quoted. The reason is given in Section IV-E.

IV. Experimental Design

Five experiments were conducted. Each varies one factor and measures its effect on the same held-out data.

A. Experiment 1: Model Comparison

Three pipelines are compared on an identical held-out split of nine photographs containing 72 instances: the classical detector with the from-scratch classifier; the fine-tuned YOLO detector using its own classification head; and a hybrid using YOLO for localisation and the from-scratch classifier for reading. The hybrid isolates which component is responsible for any difference between the first two. Hypothesis: the learned detector will localise more robustly, and the dedicated classifier will read more accurately.

B. Experiment 2: Training Data Volume

The classifier was trained with progressively more real photographs per class—one, then approximately four, then approximately eight—holding architecture fixed. Hypothesis: accuracy is limited by the number of genuinely distinct real examples rather than by model capacity, so it should rise with data volume even though the architecture is unchanged.

C. Experiment 3: Classifier Head Architecture

Ninety-five percent of the classifier's parameters lie in a single fully connected layer that maps the flattened 8x8x64 feature map to 128 units. Global average pooling is the standard remedy for such a parameter concentration [7], reducing the model to approximately 45,000 parameters. Hypothesis: with a training set dominated by synthetic augmentation of a small number of real examples, reducing this overfitting surface should improve generalisation.

D. Experiment 4: Training Duration

Training length was varied between 15 and 40 epochs at two dataset sizes, monitoring training accuracy alongside held-out accuracy to distinguish underfitting from overfitting. Hypothesis: as the real dataset grows, the training distribution becomes harder to fit and a schedule tuned on the smaller dataset will leave the model underfitted.

E. Experiment 5: Measurement Reliability

This experiment concerns the measurement instrument rather than the model. The same configuration was trained five times under different random seeds and evaluated on the same held-out data, in order to quantify how much of an observed difference between configurations could be attributed to training stochasticity alone.

V. Results

A. Detector Performance

TABLE III. YOLOv8n detection performance on the held-out split

Metric	Value
Precision	0.729
Recall	0.794
mAP@0.5	0.914
mAP@0.5:0.95	0.894

The aggregate figures are respectable for a model fine-tuned on eighteen photographs. mAP@0.5:0.95 of 0.894 sitting close to mAP@0.5 of 0.914 indicates that when the detector does find a card, it localises it tightly—unsurprising, since the objects are high-contrast quadrilaterals with unambiguous boundaries. The interesting information is in the spread across classes rather than in the mean.

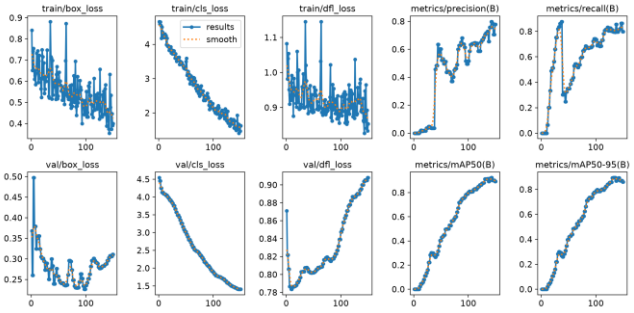


Fig. 2. YOLOv8n training and validation curves over 150 epochs. Box, classification and distribution-focal losses decrease throughout; precision, recall and mAP rise steeply after approximately epoch 30 and plateau by epoch 120.

Per-class results are markedly uneven. Fourteen classes reach recall of 1.0, but several do not. The letter F was never detected at all, giving recall of 0.0; T reaches precision 0.200 and recall 0.150; Z reaches recall 0.280; A reaches 0.333; and U and V reach 0.488 and 0.455 respectively. The classes that fail are not obviously harder to see than those that succeed, which points to training coverage rather than intrinsic difficulty—those classes appear in fewer training photographs and in fewer distinct positions within them.

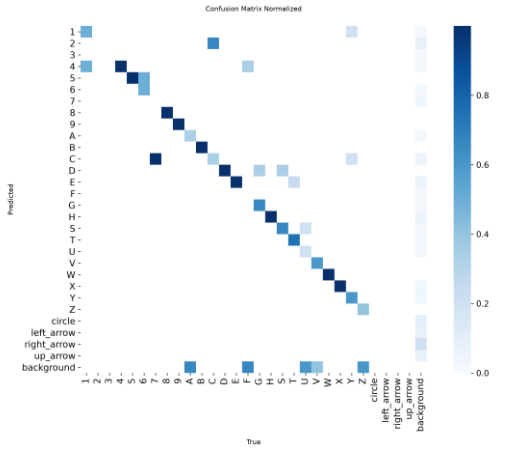


Fig. 3. Normalised confusion matrix for the fine-tuned detector on the held-out split. Off-diagonal mass is concentrated in a small number of visually similar pairs and in the background row, the latter corresponding to missed detections.



Fig. 4. Qualitative detections on a held-out photograph, with predicted class and confidence. Most symbols are detected and read correctly; missed and misread instances are discussed in the text.

B. *Pipeline Comparison*

TABLE IV. THREE-PIPELINE COMPARISON ON THE IDENTICAL HELD-OUT SPLIT (72 INSTANCES). CLS|DET IS CLASSIFICATION ACCURACY GIVEN A CORRECT DETECTION.

Pipeline	Recall	Prec.	Cls|det	Overall
Classical detect + CNN	100%	76.6%	94.4%	94.4%
YOLO (detect + classify)	84.7%	71.8%	90.2%	76.4%
Hybrid: YOLO + CNN	84.7%	71.8%	93.4%	79.2%

The hypothesis was half right and half wrong. The dedicated classifier does read more accurately than YOLO's own head: comparing rows two and three, which share identical detections, accuracy rises from 90.2% to 93.4% purely by substituting the classifier. That is a modest but real advantage, and it is what one would expect from a network specialised for a single 25-way discrimination against a lightweight head trained jointly with localisation.

The prediction about localisation was wrong, and decisively so. The classical detector found every one of the 72 instances; YOLO missed eleven. Because the hybrid inherits YOLO's localisation, that gap—not the classifier—determines the ranking. The lower precision of the classical detector, at 76.6%, reflects spurious candidate regions that survive its area and aspect filters; in a system where a downstream classifier can reject them, missing a symbol is the more costly error.

C. *Effect of Training Data Volume*

TABLE V. CLASSIFIER ACCURACY VERSUS TRAINING DATA VOLUME, MEAN AND STANDARD DEVIATION OVER FIVE SEEDS.

Real photos	Epochs	Test set	Accuracy
99 (cleaned)	30	25 inst.	68.8% ± 1.6%
104 (uncleaned)	30	25 inst.	77.6% ± 6.5%
211 (cleaned)	40	72 inst.	81.4% ± 6.5%

Accuracy rose from roughly 69% to roughly 81% as the number of real photographs per class grew from one to about eight, with the architecture held constant throughout. The comparison must be qualified honestly: the final row is measured on a larger held-out set than the first two, and epoch count differs, so these rows are not a controlled ablation of data volume alone. The middle row is also anomalous—removing corrupted training examples appeared to reduce

mean accuracy—which at that sample size is not a reliable signal and should not be read as evidence that corrupt data helps.

D. Classifier Head Architecture

TABLE VI. EFFECT OF REPLACING THE FULLY CONNECTED HEAD WITH GLOBAL AVERAGE POOLING.

Head	Parameters	Accuracy (5 seeds)
Flatten + FC(4096→128)	≈551,000	81.4% ± 6.5%
Global average pooling	≈45,000	55.6% ± 9.8%

The hypothesis was wrong by a wide margin. Removing 92% of the parameters cost 26 points of accuracy, and the smaller model was also more variable across seeds rather than less—the opposite of what a successful regularisation change produces. The explanation is that global average pooling discards where in the frame a feature activated, retaining only whether it activated. That is the correct trade for a large object appearing anywhere in a scene, but these symbols are already cropped and centred by the segmentation stage, and distinguishing S from 5, or 8 from B, depends precisely on the spatial arrangement of strokes relative to one another. The change was reverted; the reasoning and the measurement are retained in the source as a documented negative result.

E. Training Duration and Measurement Reliability

Training accuracy, not held-out accuracy, revealed that the classifier was underfitting after the dataset grew. At 15 epochs training accuracy reached only 77%; extending to 40 epochs raised it to 95.6% on the same data. A schedule adequate for one example per class was inadequate once each class had eight genuinely different examples to reconcile.

Experiment 5 produced the most methodologically important result in this work. Identical configurations, differing only in random seed, produced held-out scores ranging from 73.6% to 93.1%. Earlier in the project, single-run comparisons had appeared to show a configuration change moving accuracy from 76% to 60%; repeated under five seeds, that apparent effect was within the noise. On a held-out set of this size a single flipped instance moves the score by more than a percentage point, and training stochasticity moves it by considerably more. Every classifier figure in this report is therefore a five-seed mean, and single-run numbers are not treated as evidence.

VI. CHALLENGING SCENARIOS

Four adverse conditions were tested. Three were tested on purpose-taken photographs; the fourth emerged during dataset construction.

A. Camera Flash on a Reflective Background

A card was photographed with flash on a black rubber mat with raised reflective dimples. The classifier reported the letter D, with full confidence, for a card bearing T. Inspecting the mask actually presented to the network explains why, and exonerates the classifier: the flash blew out roughly the lower half of the card, and each dimple on the mat returned enough light to register as foreground. The segmentation stage assumes background is darker than the symbol; flash on a glossy dark object near a reflective surface violates that assumption directly. What reached the classifier was largely noise, and D was simply its least-bad interpretation. This is a segmentation failure, not a classification failure, and the distinction matters because the remedies differ.

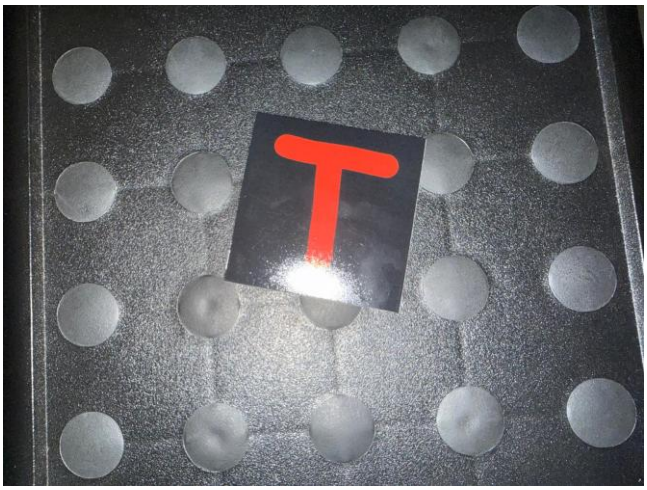


Fig. 5. Failure case. Flash illumination on a glossy card against a reflective dimpled mat. Specular highlights on both the card and the background defeat the dark-background assumption on which the segmentation stage depends.

B. Unseen Backgrounds

Three photographs were taken against backgrounds absent from all training data: a woven carpet, a household appliance with a tiled floor and a foot in frame, and the reflective mat above. The result was the reverse of expectation. The fine-tuned detector produced no detections on any of the three, even at a permissive confidence threshold, while the classical geometric rule located and correctly read two, and on the third produced no detection rather than a confident error—its area and aspect filters rejected the glare region that the single-card path had accepted.

The interpretation is that the advantage usually attributed to a learned detector—that it recognises objects by appearance rather than by a brittle hand-specified rule—is contingent on having seen enough variation to generalise. Trained on eighteen photographs spanning three surfaces, the detector learned the appearance of cards on those surfaces. The threshold rule never specialised to any surface, because it encodes a property of the cards themselves, and so transferred without difficulty.

C. Adjacent and Touching Symbols

An early photography session placed cards edge to edge. Because the classical detector groups pixels into connected components, touching cards merge into a single region and are annotated and detected as one object. This is a structural limitation of the classical approach that a learned detector does not share: YOLO's box-proposal mechanism is designed to separate adjacent instances. The practical mitigation adopted was to re-photograph with visible gaps, which is acceptable for dataset construction but would not be acceptable in deployment.

D. Rotation and Visually Similar Classes

Rotation was constrained by design. Both systems assume symbols are approximately upright, within roughly twenty-five degrees, and the augmentation used to train the classifier reflects that range. Beyond it, an unavoidable ambiguity appears: a 6 rotated by 180 degrees is indistinguishable in shape from a 9. Because colour is deliberately discarded, no shape-based system can resolve that pair under free rotation; only the deck's colour convention could, at the cost of the very shortcut the design removes.

Visually similar pairs account for most of the remaining classification errors in both systems, but the pairs differ. The from-scratch classifier confuses 1 with 2, V with W, 1 with 4 and 9 with S. The detector's own head confuses T with E repeatedly, and also 5 with F and F with 4. That the two systems fail on different pairs, given identical inputs, indicates the confusions arise from what each learned rather than from an intrinsic property of the symbols.

VII. Discussion

A. An Inappropriate Default Augmentation

Reviewing the recorded training configuration after the fact revealed a setting that should not have been left at its default. Ultralytics enables horizontal flipping with probability 0.5, together with mosaic composition, random erasing, scaling, translation and HSV jitter. For natural objects horizontal flipping is sound: a mirrored dog is still a dog. For text symbols it is not. A mirrored S is not an S, and the consequence for the arrow classes is worse than merely unhelpful: mirroring a left-pointing arrow produces an image of a right-pointing arrow that is still labelled left arrow. Half of all arrow training examples were therefore label-corrupted by construction.

Of the enabled augmentations, scaling, translation, mild HSV jitter and mosaic are defensible for this problem; random erasing is questionable for symbols whose identity depends on complete strokes; horizontal and vertical flipping should be disabled outright; and small-angle rotation, which is disabled by default, should arguably be enabled, since cards in these photographs are routinely rotated by ten to twenty degrees. This was diagnosed by inspection rather than by ablation, so it remains a well-founded hypothesis rather than a measured result. Re-training with flipping disabled is the single most immediate next experiment, and it is expected to improve both the arrow classes and overall classification.

B. Data Volume Against Model Capacity

A consistent pattern runs through the experiments. Where performance improved, it improved because the model was shown more or better data: increasing real examples per class from one to eight, and removing corrupted examples, moved the classifier substantially. Where performance was pursued through architecture instead, it did not improve: reducing the head by 92% cost 26 points, and a detector with roughly six times the classifier's parameters and a large-scale pretrained backbone nonetheless failed to generalise to unseen backgrounds that a threshold rule handled without difficulty.

This should not be read as an argument against learned detectors. It is an argument about matching model generality to data diversity. Eighteen photographs spanning three surfaces is not enough diversity for a detector to learn a surface-independent concept of the object, in the same way that one photograph per class was not enough for a classifier to learn a pose-independent concept of a symbol. The remedy in both cases is the same and is cheap here: roughly forty already-captured photographs remain unused, and adding them to the detector's training set requires no new photography.

C. Limitations

Several limitations should be stated plainly. Only one detector training run was performed, so the detector figures are single-run and, by this report's own argument, should be treated with the caution applied to the classifier's single-run numbers. The detector experiments are consequently thinner than the classifier experiments: the data-volume, duration and architecture experiments were conducted on the classifier, and equivalents for the detector remain to be run. The held-out set of nine photographs and 72 instances is small, and per-class conclusions drawn from one or two instances are weak. Finally, the classical pipeline's apparent superiority is specific to this deck and these conditions; it depends on an assumption—dark, matte cards on lighter backgrounds—that the flash experiment shows is easy to break.

D. Future Work

Four directions follow directly from the results. First, retrain the detector with flipping disabled and rotation enabled, and measure the effect on the arrow and letter classes. Second, add the forty unused photographs and re-measure both recall and out-of-distribution behaviour, testing the diagnosis in Section VII-B. Third, replace the threshold-based segmentation with a learned segmenter such as U-Net [4] or a promptable model such as Segment Anything [5], which would not depend on the dark-background assumption that the flash case defeats. Fourth, run the detector across seeds so its figures carry the same reliability as the classifier's.

VIII. Conclusion

A YOLOv8n detector was adapted by transfer learning to a 29-class symbol-detection task on a dataset captured, annotated and verified for this assignment, reaching mAP@0.5 of 0.914 and mAP@0.5:0.95 of 0.894 with uneven per-class recall. A second pipeline combining classical segmentation with a small from-scratch classifier was evaluated on identical data and, by separating detection from classification, showed that the classifier reads more accurately than the detector's own head while the detector localises less reliably than a hand-specified geometric rule—decisively so on unseen backgrounds, where it detected nothing at all.

The most durable findings are methodological. Single training runs at this data scale are not measurements: identical configurations varied by nearly twenty points across random seeds. Corrupted training data can persist undetected when there is only one example per class to define it. Failures should be diagnosed to the stage that produced them, since a confident wrong answer from a classifier may in fact be a segmentation failure. And a default augmentation setting that is standard for natural images can silently corrupt labels for symbol data. Each of these cost more to learn than the accuracy figures did, and each generalises further.

IX. AI Usage Declaration

AI tools were used in this assignment and their contribution is declared here in full.

Tool used: Anthropic Claude, accessed through the Claude Code command-line interface, used interactively throughout development.

What it was used for: assistance in writing and debugging Python code for the segmentation pipeline, the classifier, the dataset-preparation and annotation-verification scripts, the YOLO training and evaluation scripts, and the evaluation harness; assistance in diagnosing failures, including the tilted-card and edge-touching segmentation defects and the corrupted training crops; discussion of experimental design and of how to report results honestly; and drafting assistance for this report.

Which parts were AI-assisted: the majority of the source code was written with AI assistance. The experimental direction, the decisions about what to investigate, the data collection and photography, and the final interpretation of results were directed by the author. This report was drafted with AI assistance from the experimental record and then reviewed and revised by the author.

How the material was verified: every reported number in this document was produced by running the scripts in the accompanying repository, and the outputs are reproducible from it. Intermediate artefacts were inspected visually rather than accepted on trust—an audit of this kind is what revealed the nine corrupted training crops described in Section II-E, several of which had passed an automated numerical check. Claims that could not be verified by measurement are identified as hypotheses in the text, most notably the augmentation diagnosis in Section VII-A.

A separate script in the repository, identify_cards.py, queries a hosted vision-language model through the Azure OpenAI API. It was written during exploration, is not part of either pipeline evaluated here, and contributes to none of the reported results. It is retained only as a point of reference and is declared for completeness.

X. References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," version 8, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 740-755.

[4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in Proc. MICCAI, 2015, pp. 234-241.

[5] A. Kirillov et al., "Segment Anything," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2023, pp. 4015-4026.

[6] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in Proc. Int. Conf. Machine Learning (ICML), 2015, pp. 448-456.

[7] M. Lin, Q. Chen, and S. Yan, "Network in Network," in Proc. Int. Conf. Learning Representations (ICLR), 2014.

[8] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proc. Int. Conf. Learning Representations (ICLR), 2019.

[9] C. B. Barber, D. P. Dobkin, and H. Huhdanpaa, "The Quickhull Algorithm for Convex Hulls," ACM Trans. Mathematical Software, vol. 22, no. 4, pp. 469-483, 1996.

[10] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020.